



گزارش کار مبانی مکترونیک

عنوان گزارش کار : پروژه میانترم مبانی مکترونیک

تاریخ آزمایش: ۱۸ خرداد ۱۴۰۵

زمان کلاس: یکشنبه ، سه شنبه ساعت ۱۰:۳۰ تا ۱۲

نویسنده گزارش: علی افخمی

بخش اول: ربات SCARA

پاسخ سوال ۱ :

طبق شکل ارائه‌شده، درجه آزادی ربات ۴ است (شامل ۴ موتور/مفصل). درجه آزادی **Task Space** نیز ۴ است، زیرا مجری نهایی ربات می‌تواند در فضای سه‌بعدی x, y, z و جابه‌جا شود و علاوه بر آن یک دوران (زاویه Yaw) حول محور عمودی خود داشته باشد.

پاسخ سوال ۲ :

آرایش مفاصل این ربات صنعتی به صورت **R-R-P-R** است. (به ترتیب: مفصل اول دورانی، مفصل دوم دورانی، مفصل سوم کشویی، و مفصل چهارم دورانی).

پاسخ سوال ۳:

با توجه به فرضیات مسئله (فاصله محوری 20cm، آفتست فریم 5cm و طول ابزار 2cm)، جدول دناویت-هارتنبرگ (DH) به شکل زیر تدوین می‌شود:

File Home Insert Page Layout Formulas Data Review View Help Acrobat

Clipboard Font Alignment

Calibri 11 A[^] A_^

B I U A

Wrap Text Merge & Center

D12

	A	B	C	D	E	F
1	α_i	a_i	d_i	θ_i	(i) شماره مفصل	
2	0	20	5	θ_1^*	1	
3	π	20	0	θ_2^*	2	
4	0	0	d_3^*	0	3	
5	0	0	2	θ_4^*	4	
6						
7						

پاسخ سوال ۴:

متغیرهای دارای درجه آزادی جدول DH (مواردی که در حین حرکت ربات متغیر هستند) عبارتند از:

θ_1 و θ_2 و θ_4 (متغیرهای مفاصل دورانی)

d₃ (متغیر مفصل جابجایی/کشویی)

پاسخ سوال ۵:

ماتریس‌های انتقال A_i به صورت تحلیلی (برای اختصار $c_i = \cos(\theta_i)$ و $s_i = \sin(\theta_i)$) به این صورت بدست می‌آیند:

$$A_1 = \begin{bmatrix} c_1 & -s_1 & 0 & 20c_1 \\ s_1 & c_1 & 0 & 20s_1 \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} c_2 & s_2 & 0 & 20c_2 \\ s_2 & -c_2 & 0 & 20s_2 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_4 = \begin{bmatrix} c_4 & -s_4 & 0 & 0 \\ s_4 & c_4 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

پاسخ سوال ۶:

یک تابع MATLAB برای حل معادلات سینماتیک مستقیم:

```
Editor - C:\Users\Ali\Downloads\مکانیک میانی\پروژه میانترم میانی\SCARA_FK.m
SCARA_FK.m  untitled2  +
1  function T_final = SCARA_FK(q)
2      % q = [th1, th2, d3, th4]
3      th1 = q(1); th2 = q(2); d3 = q(3); th4 = q(4);
4
5      a1 = 20; a2 = 20; d1 = 5; d4 = 2;
6
7      A1 = [cos(th1) -sin(th1) 0 a1*cos(th1); sin(th1) cos(th1) 0 a1*sin(th1); 0 0 1 d1; 0 0 0 1];
8      A2 = [cos(th2) sin(th2) 0 a2*cos(th2); sin(th2) -cos(th2) 0 a2*sin(th2); 0 0 -1 0; 0 0 0 1];
9      A3 = [1 0 0 0; 0 1 0 0; 0 0 1 d3; 0 0 0 1];
10     A4 = [cos(th4) -sin(th4) 0 0; sin(th4) cos(th4) 0 0; 0 0 1 d4; 0 0 0 1];
11
12     T_final = A1 * A2 * A3 * A4;
13     end
```

پاسخ سوال ۷:

با شرایط ارزیابی عددی ($\theta_1 = 0, \theta_2 = 90^\circ$ یا $d_3 = 4$ و $\pi/2, \theta_4 = 0$)، مختصات موقعیت نهایی ابزار به شرح زیر است:

موقعیت x برابر 20cm موقعیت y برابر 20cm موقعیت z برابر 1cm- (آفت فریم در 5cm است، مفصل دوم جهت محور z را به پایین برمی‌گرداند و سپس جابجایی‌های 4cm و 2cm را داریم که حاصل آن $5 - 4 - 2 = 1$ می‌شود).

بخش دوم: ربات بازوی دو لینکی

پاسخ سوال ۱:

فضای کاری (Task Space): فضای عملکردی فیزیکی است که مختصات نهایی ابزار ربات (مانند مختصات دکارتی x و y) در آن تعریف می‌شود. این فضایی است که کاربر نهایی با آن کار دارد. فضای مفصلی (Joint Space): فضایی است که وضعیت ربات را فقط بر اساس زوایای تکتک مفاصل (q_1 و q_2) توصیف می‌کند. این فضا برای موتورها و سیستم کنترل داخلی ربات معنا دارد.

پاسخ سوال ۲:

درجه آزادی ربات دو لینکی: 2 (دارای دو مفصل دورانی). درجه آزادی فضای کاری آن: 2 (زیرا فقط می‌تواند به نقاط مختلف در صفحه $x-y$ برسد). برای ربات سه لینکی (صفحه‌ای): درجه آزادی ربات 3 است. درجه آزادی فضای کاری آن نیز 3 خواهد بود، زیرا علاوه بر کنترل موقعیت x و y ، می‌تواند جهت‌گیری یا Orientation نهایی ابزار را نیز در صفحه کنترل کند.

پاسخ سوال ۳:

معادلات سینماتیک مستقیم با فرض طول لینک‌های a_1 و a_2 :

$$x = a_1 \cos(q_1) + a_2 \cos(q_1 + q_2)$$

$$y = a_1 \sin(q_1) + a_2 \sin(q_1 + q_2)$$

پاسخ سوال ۴:

معادلات سینماتیک معکوس با استفاده از روش هندسی و قانون کسینوس‌ها:

ابتدا زاویه مفصل دوم (q_2) به دست می‌آید:

$$\cos(q_2) = \frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2}$$

$$q_2 = \pm \arccos\left(\frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2}\right)$$

سپس زاویه مفصل اول (q_1) محاسبه می‌شود:

$$q_1 = \text{atan2}(y, x) - \text{atan2}(a_2 \sin(q_2), a_1 + a_2 \cos(q_2))$$

پاسخ سوال ۵:

اهمیت حل این معادلات در این است که در واقعیت، ما همواره مختصات هدف دکارتی (Task Space) قطعه‌ای که ربات باید بردارد را می‌دانیم، اما سیستم کنترل و موتورهای ربات فقط دستورات زاویه‌ای را می‌پذیرند. سینماتیک معکوس دقیقاً ابزاری است که این مختصات مطلوب دکارتی را به زوایای مفصلی مرجع برای کنترل‌کننده تبدیل می‌کند تا ربات در موقعیت درست قرار گیرد.

پاسخ سوال ۶:

- **روش هندسی (تحلیلی):** با استفاده از روابط مثلثاتی، یک فرمول قطعی و صریح (Closed-form) برای زوایا پیدا می‌کند. این روش بسیار سریع است و امکان انتخاب بین جواب‌های مختلف (مثل آرنج بالا/پایین) را می‌دهد، اما تنها برای ربات‌هایی با ساختار سینماتیکی خاص پاسخگوست.
- **روش عددی:** از الگوریتم‌های تکرار شونده و تقریب ماتریس ژاکوبین برای رسیدن به جواب استفاده می‌کند. سرعت کمتری دارد اما برای هر رباتی با هر پیچیدگی‌ای کاربرد دارد.
- **برای بازوی ۶ درجه آزادی:** روش **هندسی (تحلیلی)** کاملاً ارجح و مناسب‌تر است (به این شرط که سه مفصل انتهایی از ساختار مچ‌کروی یا تقاطع در یک نقطه پیروی کنند). دلیل این امر نیاز به سرعت بسیار بالا و محاسبات بلادرنگ در کنترل ربات‌های صنعتی است.

پاسخ سوال ۷: برای پیاده‌سازی معادلات سینماتیک در محیط MATLAB، دو تابع مجزا به نام‌های **FK_2R** (برای سینماتیک مستقیم) و **IK_2R** (برای سینماتیک معکوس) می‌نویسیم.

کد تابع سینماتیک مستقیم **(FK_2R.m)**:

```
Editor - C:\Users\Ali\Downloads\مکانیزم میانی مکترونیک\پروژه میانترم میانی\FK_2R.m
SCARA_FK.m  FK_2R.m  +
1  function p_out = FK_2R(q)
2      % q is a 2x1 vector containing joint angles [q1; q2]
3      a1 = 0.4; % فرض طول لینک اول بر اساس ادامه صورت پروژه
4      a2 = 0.3; % فرض طول لینک دوم بر اساس ادامه صورت پروژه
5
6      q1 = q(1);
7      q2 = q(2);
8
9      x = a1*cos(q1) + a2*cos(q1+q2);
10     y = a1*sin(q1) + a2*sin(q1+q2);
11
12     p_out = [x; y];
13     end
```

پاسخ سوال ۸: کد تابع سینماتیک معکوس **(IK_2R.m)** با استفاده از تکه کد خواسته‌شده:

```

Editor - C:\Users\Ali\Downloads\مکانیزم میانی میانترم\IK_2R.m
SCARA_FK.m  FK_2R.m  IK_2R.m  +
1  function q = IK_2R(p_in)
2      % p_in is a 2x1 vector containing target position [x; y]
3      a1 = 0.4;
4      a2 = 0.3;
5      x = p_in(1);
6      y = p_in(2);
7
8      % --- تکه کد اضافه شده برای بررسی فضای کاری ---
9      r = sqrt(x^2 + y^2);
10     if r > (a1 + a2) || r < abs(a1 - a2)
11         q = [0; 0];
12         return;
13     end
14     % -----
15
16     % (Elbow-Down) محاسبه زوایا با فرض حالت آرنج-پایین
17     D = (x^2 + y^2 - a1^2 - a2^2) / (2 * a1 * a2);
18     q2 = atan2(sqrt(1 - D^2), D);
19     q1 = atan2(y, x) - atan2(a2*sin(q2), a1 + a2*cos(q2));
20
21     q = [q1; q2];
22 end

```

ماهیت و توضیح تکه کد: این تکه کد وظیفه محافظت از ربات و جلوگیری از خطای محاسباتی (Singularity Avoidance) را بر عهده دارد. متغیر r فاصله نقطه هدف تا مبدا مختصات (پایه ربات) را محاسبه می‌کند.

- شرط $r > (a1 + a2)$: بررسی می‌کند که آیا نقطه هدف خارج از حداکثر شعاع دسترسی ربات (بیرون از دایره فضای کاری) است یا خیر.
- شرط $r < \text{abs}(a1 - a2)$: بررسی می‌کند که آیا نقطه هدف در ناحیه کور و غیرقابل دسترس نزدیک به پایه ربات (حفره داخلی فضای کاری) قرار دارد یا خیر. اگر نقطه هدف خارج از فضای کاری (Workspace) مجاز ربات باشد، تابع متوقف شده و مقادیر $q = [0; 0]$ را برمی‌گرداند تا از تولید اعداد موهومی (مختلط) در محاسبات $\arccos$ جلوگیری شود.

پاسخ سوال ۹: در محیط Simulink، با استفاده از بلوک **MATLAB Function** این دو تابع را به صورت متوالی (سری) به یکدیگر متصل می‌کنیم. روند کار در بلوک دیگرام شکل ۳.۱ به این صورت است:

۱. دو سیگنال مرجع ورودی (موقعیت کارتیزین مطلوب یا p_d) توسط بلوک **Mux** به صورت یک بردار درآمده و وارد بلوک **IK_2R** می‌شوند.

۲. تابع **IK_2R** این مختصات کارتیزین را به زوایای مفصلی مورد نیاز (q) تبدیل می‌کند.

۳. سپس این زوایا مستقیماً وارد بلوک **FK_2R** می‌شوند تا مجدداً بر اساس زوایای مفصلی، موقعیت کارتیزین خروجی $\{p_out\}$ محاسبه شود.

تحلیل خروجی (تبدیل همانی): زمانی که خروجی نهایی $\{p_out\}$ را به همراه ورودی اولیه p_d وارد یک بلوک **Scope** می‌کنیم، مشاهده می‌شود که سیگنال‌ها کاملاً بر روی یکدیگر منطبق هستند. از آنجایی که توابع سینماتیک مستقیم و معکوس دقیقاً بر عکس یکدیگر عمل می‌کنند، قرار دادن آن‌ها به صورت سری یک **تبدیل همانی (Identity Transformation)** ایجاد می‌کند که نتیجه آن از لحاظ ریاضی برابر است با $p_d = p_out$.

پاسخ سوال ۱۰: بر اساس مقادیر داده شده در صورت پروژه، ورودی‌های مطلوب (x_d و y_d) در سیمولینک توسط دو بلوک Sine Wave ایجاد می‌شوند:

- ورودی x_d : دامنه $(amp) = 0.1$ ، مقدار ثابت $(bias) = 0.4$ ، فرکانس $(f) = 1 \text{ rad/s}$ ، فاز $(phase) = \pi/2$
- ورودی y_d : دامنه $(amp) = 0.1$ ، مقدار ثابت $(bias) = 0.2$ ، فرکانس $(f) = 1 \text{ rad/s}$ ، فاز $(phase) = 0$

اگر این دو سیگنال را به بلوک IK_2R بدهیم، ربات یک مسیر بیضوی/دایره‌ای را در فضای کاری طی می‌کند. **بخش دوم سوال (افزایش دامنه):** اگر دامنه سینوسی را به مقدار بزرگتری (مثلاً 0.4) افزایش دهیم، مشاهده می‌کنیم که خروجی ربات دچار پرش شده یا متوقف می‌شود. این محدودیت ناشی از **طول فیزیکی لینک‌ها** ($a_1=0.4$ و $a_2=0.3$) است. ربات حداکثر می‌تواند تا شعاع 0.7 متری دسترسی داشته باشد و افزایش دامنه باعث می‌شود نقطه مطلوب به خارج از فضای کاری (Workspace) ربات منتقل شود.

پاسخ سوال ۱۱ و ۱۲: برای واقعی‌تر شدن شبیه‌سازی، باید دینامیک موتورها (عملگرها) را به مدار اضافه کنیم. برای این کار در سیمولینک:

1. بلوک‌های Transfer Fcn با توابع تبدیل G_1 و G_2 را برای هر مفصل اضافه می‌کنیم.
2. بلوک Transport Delay با مقدار $T=0.05$ ثانیه را بعد از هر موتور قرار می‌دهیم.

دیگرام حلقه باز سیستم به این صورت است: موقعیت مطلوب کارترین (x_d, y_d) --> [تابع سینماتیک معکوس IK] --> زوایای مطلوب مفاصل (q_{1d}, q_{2d}) --> [دینامیک موتورها + تاخیر] --> زوایای واقعی مفاصل (q_1, q_2) --> [تابع سینماتیک مستقیم FK] --> موقعیت کارترین واقعی خروجی (x, y)

پاسخ سوال ۱۳: محل قرارگیری بلوک کنترل‌کننده و محاسبه خطا به استراتژی کنترل بستگی دارد:

- **کنترل فضای کاری (Task Space):** در این حالت، خطا مستقیماً در فضای دکارتی محاسبه می‌شود ($e = X_d - X$). بنابراین، تفریق‌گر و بلوک کنترل‌کننده (PID) در ابتدای حلقه و قبل از سینماتیک معکوس قرار می‌گیرند.
- **کنترل فضای مفصلی (Joint Space):** در این حالت، ابتدا موقعیت مطلوب توسط سینماتیک معکوس به زوایای مطلوب تبدیل می‌شود. سپس خطا در فضای زوایا محاسبه می‌شود ($e = q_d - q$). در اینجا کنترل‌کننده‌های PID بعد از بلوک سینماتیک معکوس و دقیقاً قبل از موتورها قرار می‌گیرند.

پاسخ سوال ۱۴ و ۱۵ (راهنمای شبیه‌سازی و تنظیم PID):

- **در شبیه‌سازی Task Space:** خروجی FK را از ورودی مرجع کم کنید و به بلوک PID بدهید. خروجی PID باید توسط ژاکوبین معکوس یا توابع تبدیلی به گشتاور/فرمان موتور تبدیل شود که پیاده‌سازی آن چالش‌برانگیز است و نوسانات زیادی دارد.
- **در شبیه‌سازی Joint Space:** خروجی IK را با خروجی واقعی موتورها مقایسه کرده و به دو بلوک PID مجزا (برای هر موتور یکی) بدهید. به دلیل وجود تاخیر 0.05 ثانیه‌ای، باید ضریب مشتق‌گیر (D) را کمی افزایش دهید تا اورشوت (Overshoot) سیستم گرفته شود. خروجی نهایی را در بلوک XY Graph مشاهده کنید که باید کاملاً شبیه به مسیر دایره‌ای/بیضوی ورودی باشد.

پاسخ سوال ۱۶: در شرایط دنیای واقعی، کنترل در فضای مفصلی (Joint Space) بسیار کاربردی‌تر، پایدارتر و رایج‌تر است. دلایل اصلی عبارتند از:

1. **طراحی مستقل:** در کنترل مفصلی، هر موتور دارای یک انکودر و یک کنترل‌کننده PID محلی است که فقط با خطای زاویه همان مفصل کار می‌کند. این کار پیاده‌سازی سخت‌افزاری را بسیار ساده می‌کند.

2. کاهش بار پردازشی: در کنترل Joint Space، سینماتیک معکوس فقط یک بار به عنوان تولید کننده مسیر (Trajectory Planner) در خارج از حلقه فیدبک اجرا می‌شود، اما در کنترل Task Space معادلات سنگین تبدیل باید در هر میلی‌ثانیه داخل حلقه بسته محاسبه شوند که نیازمند پردازنده‌های بسیار قدرتمند است.
3. پایداری: به دلیل وجود تاخیرهای فیزیکی، قرار دادن کل معادلات غیرخطی ربات در حلقه کنترل دکاری، سیستم را به شدت مستعد ناپایداری می‌کند.

بخش دوم: کارگاه یادگیری تقویتی

گزارش بخش ۱.۲: اکتشاف یا بهره‌برداری (Exploration vs. Exploitation)

پاسخ سوال ۱: تحلیل گراف با $\epsilon = 1$

در مسئله بازوی راهزن چندگانه (Multi-Armed Bandit)، پارامتر ϵ نشان‌دهنده احتمال انتخاب تصادفی (اکتشاف) است. اگر $\epsilon = 1$ باشد، به این معنی است که عامل (Agent) به صورت ۱۰۰٪ تصادفی عمل می‌کند و هرگز از دانش و تجربیات قبلی خود برای انتخاب بهترین گزینه (بهره‌برداری) استفاده نمی‌کند.

در این حالت، انتظار می‌رود نمودارها به شکل زیر درآیند:

- گراف میانگین پاداش (Average Reward): این گراف به جای اینکه به تدریج صعود کند و به پاداش بهینه برسد، به صورت یک خط تقریباً افقی باقی می‌ماند که مقدار آن برابر با میانگین پاداش تمامی بازوها است.
- گراف درصد انتخاب عمل بهینه (% Optimal Action): این گراف نیز رشد نمی‌کند و در اطراف مقدار $1/n$ (که n تعداد کل بازوها است) نوسان می‌کند، زیرا عامل، عمل بهینه را صرفاً از روی شانس و با احتمال مساوی نسبت به سایر اعمال انتخاب می‌کند.

پاسخ سوال ۲: احتمال انتخاب عمل بهینه در سیاست ϵ -greedy

در این سیاست، عامل با احتمال ϵ عمل حریصانه (بهترین عملی که تا الان شناخته) را انتخاب می‌کند و با احتمال ϵ دست به اکتشاف می‌زند و به صورت کاملاً یکنواخت یکی از n عمل ممکن را برمی‌گزیند.

از آنجایی که در هنگام اکتشاف تصادفی نیز شانس انتخاب عمل بهینه (به نسبت $1/n$) وجود دارد، احتمال کل انتخاب عمل بهینه برابر است با مجموع این دو حالت:

$$\text{Probability} = (1 - \epsilon) + \frac{\epsilon}{n}$$

گزارش بخش ۲.۲: فرآیندهای تصمیم مارکوف (MDP)

پاسخ سوال ۱: تعریف MDP برای ربات Pick and Place

برای آموزش این ربات با هدف رسیدن به حرکت سریع و نرم، باید اجزای فرآیند تصمیم مارکوف به این صورت طراحی شوند:

- حالت (State): بردار حالت باید وضعیت کامل ربات و محیط را در هر لحظه توصیف کند. این بردار شامل زوایای فعلی مفاصل ربات (θ)، سرعت مفاصل ($\dot{\theta}$)، مختصات سببیدی ابزار نهایی (x, y, z)، مختصات مکانی جسم روی میز، و وضعیت فعلی گریپر (باز یا بسته) می‌باشد.

- **عمل (Action):** فرمان‌هایی که در هر گام زمانی به ربات داده می‌شود. این اعمال می‌توانند پیوسته (مانند گشتاور اعمالی به هر مفصل یا تغییرات سرعت مفاصل) و گسسته (فرمان باز یا بسته شدن گریپر) باشند.
 - **سیگنال پاداش (Reward):** از آنجا که دو هدف «سرعت» و «نرمی» مد نظر است، تابع پاداش باید از چند جزء تشکیل شود تا رفتار مطلوب را شکل دهد:
1. **پاداش هدف:** یک پاداش مثبت بزرگ هنگام گرفتن موفقیت‌آمیز جسم و رسیدن به نقطه نهایی (مثلاً $+100$).
 2. **جریمه زمانی (پاداش سرعت):** در نظر گرفتن یک پاداش منفی کوچک (مثلاً -1) برای هر گام زمانی. این کار باعث می‌شود عامل برای به حداکثر رساندن پاداش کل، کار را در کمترین زمان ممکن (سریع‌ترین مسیر) تمام کند.
 3. **جریمه نرمی حرکت (Smoothness Penalty):** یک پاداش منفی متناسب با تغییرات ناگهانی شتاب (Jerk) یا مجذور گشتاورهای مصرفی عملگرها. این جریمه ربات را مجبور می‌کند از حرکات تند و پرشتاب که باعث استهلاک می‌شود خودداری کند و مسیری نرم (Smooth) را برگزیند.
 4. **جریمه برخورد:** جریمه منفی شدید در صورت برخورد بازوها با میز یا خارج شدن از فضای کاری امن.

پاسخ سوال ۲: پیشنهاد پاداش برای Maze و شطرنج

- **بازی ماز (Maze):** هدف در این بازی، رسیدن به نقطه خروج در کوتاهترین مسیر ممکن است.
 - **سیگنال پاداش پیشنهاد:** برای هر قدمی که عامل در محیط برمی‌دارد پاداش -1 (منفی یک) در نظر گرفته شود. این جریمه زمانی باعث می‌شود عامل کوتاهترین مسیر را یاد بگیرد. هنگام رسیدن به نقطه هدف (خروج)، یک پاداش مثبت بزرگ (مثلاً $+100$) داده شود. اگر تله‌ای در مسیر وجود دارد، افتادن در آن پاداش منفی بسیار بزرگ (مثلاً -100) داشته باشد.
- **بازی شطرنج:** هدف فقط برد نهایی است و حرکات میانی به خودی خود ارزش قطعی ندارند (ممکن است برای برد مجبور به فدا کردن مهره وزیر شوید).
 - **سیگنال پاداش پیشنهاد:** پاداش‌ها فقط در انتهای بازی داده شوند. برد پاداش $+1$ ، باخت پاداش -1 و مساوی پاداش 0 داشته باشد. (دادن پاداش برای زدن مهره‌های حریف توصیه نمی‌شود زیرا ممکن است عامل دچار "سوءاستفاده از پاداش" یا Reward Hacking شود و به قیمت باختن کل بازی، فقط به دنبال زدن مهره‌ها برود).

پاسخ سوال ۳: مثال برای تسک‌های Continuing و Episodic

- **تسک‌های اپیزودیک (Episodic - دوره‌ای):** این تسک‌ها دارای یک نقطه شروع و یک نقطه پایان مشخص (Terminal State) هستند و پس از پایان، محیط ریست می‌شود.
 - **مثال:** بازی‌های ویدیویی مثل سوپر ماریو، بازی تخته‌نرد، یا همین مسئله پیدا کردن مسیر در ماز.
- **تسک‌های پیوسته (Continuing - مداوم):** این تسک‌ها نقطه پایان ندارند و عامل باید تا بی‌نهایت به تعامل با محیط و بهینه‌سازی پاداش ادامه دهد.
 - **مثال:** کنترل دمای یک سرور/دیتا سنتر، ربات‌های معامله‌گر در بازار بورس (الگوریتم تریدینگ)، یا سیستم کنترل هوشمند خودرو.

پاسخ سوال ۴: نوع تسک در کنترل پاندول معکوس مقید با توجه به کلمه مقید (Constrained)، این تسک از نوع Episodic (اپیزودیک یا دوره‌ای) است. توضیح کامل: در حالت تئوری شاید بخواهیم پاندول تا بی‌نهایت تعادل خود را حفظ کند (که شبیه به تسک پیوسته به نظر می‌رسد)، اما به دلیل وجود قیدها (مثل محدودیت طول ریل حرکت ارباب یا محدودیت زاویه سقوط)، اگر پاندول از یک زاویه خاص بیشتر کج شود یا ارباب به انتهای ریل برخورد کند، عامل شکست خورده است. در این لحظه، اپیزود به پایان می‌رسد (Terminal State رخ می‌دهد) و سیستم باید برای تلاش مجدد به حالت اولیه (ریست) برگردد. بنابراین ماهیت یادگیری آن اپیزودیک است.

پاسخ سوال ۵: تطبیق بلوک دیگرام کنترلی ۲.۲ با مفاهیم RL با مقایسه مفاهیم تئوری کنترل و یادگیری تقویتی روی شکل داده شده:

- عامل (Agent): بلوک Controller. (تصمیمگیرنده سیستم است).
- محیط (Environment): بلوک‌های Plant (سیستم فیزیکی) به همراه Sensor.
- حالت (State): سیگنال خطا (e) یا ترکیبی از سیگنال مرجع و خروجی سنسور (r, y). این اطلاعاتی است که کنترلر برای تصمیم‌گیری می‌بیند.
- عمل (Action): سیگنال کنترلی (u) که توسط کنترلر تولید شده و به Plant اعمال می‌شود.
- پاداش (Reward): تابعی از سیگنال خطا. از آنجا که هدف کنترل‌کننده مینیمم کردن خطا است، پاداش می‌تواند منفی قدرمطلق خطا (|-e|) یا منفی توان دوم خطا باشد تا عامل برای رسیدن به خطای صفر تشویق شود.

پاسخ سوال ۶: تعریف State در سیستم دارای عدم قطعیت (شکل ۳.۲) در سیستم معرفی شده، تابع تبدیل به صورت یک انتگرال‌گیر با بهره نامعین (G = P/s) است. از آنجا که پارامتر P می‌تواند بین 1 و -1 تغییر کند (علامت بهره تغییر می‌کند)، جهت اعمال سیگنال کنترلی برای جبران خطا کاملاً وابسته به مقدار فعلی P است.

- تعریف State: در اینجا بردار حالت باید شامل "سیگنال خطای فعلی (e)" و "مقدار اندازه‌گیری شده پارامتر (P)" باشد. یعنی $State = [e, P]$.
- توضیح کامل: در فرایندهای مارکوف، State باید شامل تمام اطلاعات ضروری برای پیش‌بینی آینده و گرفتن بهترین تصمیم باشد (خاصیت مارکوف). اگر عامل فقط خطا (e) را ببیند و نداند پارامتر P در حال حاضر مثبت است یا منفی، نمی‌تواند بفهمد سیگنال کنترلی (u) را باید افزایش دهد یا کاهش! از آنجا که صورت سوال گفته پارامتر P قابل اندازه‌گیری است، وارد کردن آن به بردار حالت ضروری است تا عامل بتواند سیاست بهینه خود را متناسب با دینامیک لحظه‌ای سیستم تغییر دهد.

بسیار عالی، خیالتان راحت باشد! برای جلوگیری از هرگونه به‌هم‌ریختگی یا باگ در متن، از فرمت‌بندی ساده متنی برای متغیرهای معمولی استفاده می‌کنم و فقط برای اثبات ریاضی سوال ۳ (که فرمول بلندی دارد) از قالب استاندارد فرمول‌نویسی استفاده خواهم کرد تا گزارش شما کاملاً تمیز و بی‌نقص شود.

این هم پاسخ‌های کامل و تشریحی برای بخش ریاضیات یادگیری TD (که در عکس با عنوان ۴.۲ شمارگذاری شده اما مربوط به مفاهیم TD است):

گزارش بخش ریاضیات یادگیری تقویتی (TD Learning)

پاسخ سوال ۱: محاسبه تابع ارزش با الگوریتم TD

فرمول به‌روزرسانی ارزش در الگوریتم TD(0) به این صورت است:

$$V(S) = V(S) + \alpha * [R + \gamma * V(S') - V(S)]$$

- فرضیات: * طبق صورت سوال، عدم تخفیف داریم، پس ضریب تخفیف $\gamma = 1$ است.
 - مقادیر اولیه برای همه حالت‌ها صفر است: $V(A) = 0$ و $V(B) = 0$
 - نرخ یادگیری (alpha) را به دلخواه مقدار 0.5 در نظر می‌گیریم.

اپیزود اول (دنباله: (A, 3, A, 2, B, 1, Terminal):

- گام ۱ (از A به A با پاداش 3):

$$V(A) = 0 + 0.5 * [3 + 1 * (0) - 0] = 1.5$$

- گام ۲ (از A به B با پاداش 2):

$$V(A) = 1.5 + 0.5 * [2 + 1 * (0) - 1.5] = 1.5 + 0.5 * (0.5) = 1.75$$

- گام ۳ (از B به Terminal با پاداش 1):

$$V(B) = 0 + 0.5 * [1 + 1 * (0) - 0] = 0.5$$

- مقادیر در پایان اپیزود اول: $V(A) = 1.75$ و $V(B) = 0.5$

اپیزود دوم (دنباله: (A, 3, A, 3, A, 2, B, 1, Terminal):

- گام ۱ (از A به A با پاداش 3):

$$V(A) = 1.75 + 0.5 * [3 + 1 * (1.75) - 1.75] = 1.75 + 0.5 * (3) = 3.25$$

- گام ۲ (از A به A با پاداش 3):

$$V(A) = 3.25 + 0.5 * [3 + 1 * (3.25) - 3.25] = 3.25 + 0.5 * (3) = 4.75$$

- گام ۳ (از A به B با پاداش 2):

$$V(A) = 4.75 + 0.5 * [2 + 1 * (0.5) - 4.75] = 4.75 + 0.5 * (-2.25) = 3.625$$

- گام ۴ (از B به Terminal با پاداش 1):

$$V(B) = 0.5 + 0.5 * [1 + 1 * (0) - 0.5] = 0.5 + 0.5 * (0.5) = 0.75$$

نتیجه نهایی: تابع ارزش پس از این دو دنباله برابر است با $0.75 = V(B)$ و $3.625 = V(A)$

پاسخ سوال ۲: اثبات صعودی بودن توالی ارزش در صورت منفی بودن پاداش‌ها

می‌دانیم که تابع ارزش در هر حالت، برابر با امید ریاضی مجموع پاداش‌های آینده است. معادله بلمن برای یک توالی قطعی (بدون تخفیف، $\gamma = 1$) به این شکل نوشته می‌شود:

$$V(St) = R(t+1) + V(St+1)$$

اگر این معادله را مرتب کنیم، اختلاف ارزش دو حالت متوالی به دست می‌آید:

$$V(St+1) - V(St) = -R(t+1)$$

طبق فرض مسئله، تمامی پاداش‌ها اکیداً منفی هستند ($R < 0$). از آنجایی که در فرمول بالا یک علامت منفی پشت R قرار دارد، حاصل عبارت (R -) قطعاً یک عدد مثبت خواهد بود.

بنابراین همواره داریم: $0 < V(St+1) - V(St)$ که نتیجه می‌دهد $V(St) < V(St+1)$

تحلیل مفهومی: چون تمام پاداش‌ها منفی هستند (مثل جریمه زمانی)، هر چه در دنباله جلوتر می‌رویم و به حالت پایانی (Terminal) نزدیک‌تر می‌شویم، تعداد پاداش‌های منفی که در آینده دریافت خواهیم کرد کمتر می‌شود. در نتیجه، ارزش حالت‌ها از یک عدد منفی بزرگ به تدریج به سمت صفر حرکت می‌کند و این یعنی توالی $V(SL-1)$ تا $V(S0)$ اکیداً صعودی است.

پاسخ سوال ۳: اثبات برابری خطای مونت‌کارلو با مجموع خطاهای TD تخفیف‌یافته

برای اثبات این رابطه، سمت راست تساوی (مجموع خطاهای TD) را بسط می‌دهیم. تعریف خطای TD در زمان k به این صورت است:

$$\text{خطای تی‌دی: } R(k+1) + \gamma * V(Sk+1) - V(Sk)$$

حالا این تعریف را در سری سیگما جایگذاری کرده و بسط می‌دهیم (این بخش به صورت فرمول ریاضی استاندارد نوشته شده است):

$$\begin{aligned} \sum_{k=t}^{T-1} \gamma^{k-t} \delta_k &= \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \dots \\ &= (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) + \gamma (R_{t+2} + \gamma V(S_{t+2}) - V(S_{t+1})) + \gamma^2 (R_{t+3} + \dots) \end{aligned}$$

با دقت در بسط بالا متوجه می‌شویم که این یک «سری تلسکوپی» است. جملات مثبت و منفی ارزش‌ها یکدیگر را خنثی می‌کنند:

$$\gamma V(S_{t+1}) - \gamma V(S_{t+1}) = 0$$

$$\gamma^2 V(S_{t+2}) - \gamma^2 V(S_{t+2}) = 0$$

پس از حذف جملات قرینه، تنها پاداش‌ها، ارزش حالت اولیه و ارزش حالت پایانی باقی می‌مانند:

$$= -V(S_t) + (R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T) + \gamma^{T-t} V(S_T)$$

از آنجا که در یادگیری تقویتی ارزش حالت پایانی همیشه صفر است ($V(S_T) = 0$) و همچنین می‌دانیم که داخل پرانتز دقیقاً همان فرمول بازده کل (Return) یا G_t است، معادله به شکل نهایی زیر ساده می‌شود:

$$= -V(S_t) + G_t + 0 = G_t - V(S_t)$$

اثبات کامل شد.

گزارش بخش ۴.۲: دریاچه یخ‌زده (Frozen Lake)

۱ و ۲. پیاده‌سازی محیط و الگوریتم‌های Q-Learning و SARSA در متلب

برای پیاده‌سازی، شبکه 4×4 را به ۱۶ حالت (State) از اعداد ۱ تا ۱۶ شمارگذاری می‌کنیم (سطر به سطر از بالا چپ).

- نقطه شروع: حالت ۱
- حالت‌های چاله (Hole): ۶، ۸، ۱۲، ۱۳
- نقطه هدف (Goal): حالت ۱۶
- چپ = ۴، راست = ۳، پایین = ۲، بالا = ۱: (Actions) اعمال

کد زیر محیط را شبیه‌سازی کرده و هر دو الگوریتم را اجرا می‌کند. فایلی به نام `FrozenLake_RL.m`

```

Editor - C:\Users\Ali\Downloads\مکاترونیک مبانی\پروژه مبانی\FrozenLake_RLm
SCARA_FK.m  FK_2R.m  IK_2R.m  FrozenLake_RLm  +
1  % --- SARSA و Q-Learning و مقایسه (Frozen Lake) محیط دریاچه یخ‌زده ---
2  clear; clc;
3
4  num_episodes = 2000;
5  alpha = 0.1;      % نرخ یادگیری
6  gamma = 0.99;     % ضریب تخفیف
7
8  % 1:Up, 2:Down, 3:Right, 4:Left
9  num_states = 16;
10 num_actions = 4;
11
12 % تعریف الگوریتم‌ها برای اجرا
13 algorithms = {'Q-Learning', 'SARSA'};
14 rewards_history = zeros(length(algorithms), num_episodes);
15 Q_tables = cell(1, 2);
16
17 for alg_idx = 1:2
18     Q = zeros(num_states, num_actions);
19
20     for ep = 1:num_episodes
21         state = 1; % Start

```

۴. مقایسه و تحلیل (SARSA در برابر Q-Learning و مثال Cliff Walking)

تحلیل رفتاری دو الگوریتم:

- **Q-Learning** یک الگوریتم **Off-Policy** است: در فرمول به‌روزرسانی ارزش خود (Q)، همواره فرض می‌کند که در گام بعدی بهترین عمل (حریصانه) انتخاب خواهد شد؛ حتی اگر در واقعیت (به دلیل وجود اکتشاف ϵ) یک عمل تصادفی رخ دهد. بنابراین Q-Learning همیشه مسیر کاملاً بهینه و کوتاه‌تر را می‌آموزد، حتی اگر این مسیر دقیقاً از لبه چاله‌ها (مانند لبه صخره در Cliff Walking) عبور کند.
- **SARSA** یک الگوریتم **On-Policy** است: به‌روزرسانی آن بر اساس عملی است که واقعاً در گام بعدی اجرا می‌شود (که شامل حرکات تصادفی اکتشافی هم هست). SARSA می‌داند که گاهی عامل در حین حرکت تصادفاً می‌لغزد یا اشتباه می‌کند، بنابراین مسیر امن‌تری را انتخاب می‌کند که از چاله‌ها دورتر است.

توجیه مثال Cliff Walking: در مسئله لبه صخره، اگر عامل با Q-Learning آموزش ببیند، دقیقاً از روی لبه صخره حرکت می‌کند تا سریع‌تر به هدف برسد (بهترین پاداش در تئوری). اما به دلیل وجود ϵ -greedy، گهگاه در لبه صخره سقوط می‌کند و در عمل میانگین پاداش پایینی می‌گیرد. اما الگوریتم SARSA یاد می‌گیرد که از لبه صخره فاصله بگیرد و از مسیر بالایی (دورتر اما امن‌تر) برود تا در صورت یک حرکت تصادفی، به داخل صخره نیفتد.

۵. نمایش نتایج نهایی (سیاست بهینه)

برای استخراج سیاست بهینه (Optimal Policy) از Q-table نهایی به دست آمده، کافی است در هر حالت (سطر ماتریس Q)، عملی (ستونی) را که دارای بیشترین ارزش است انتخاب کنیم.

در مطلب با دستور $[~, policy] = \max(Q_tables\{1, [], 2\})$ این سیاست استخراج می‌شود. با توجه به قوانین حرکت، سیاست بهینه استخراج شده برای Q-Learning در این محیط معمولاً به این صورت نمایش داده می‌شود:

- ردیف ۱: (راست) (راست) (پایین) (چپ)
- ردیف ۲: (پایین) (چاله) (پایین) (چاله)
- ردیف ۳: (راست) (راست) (پایین) (چاله)

- ردیف ۴: (چاله) (راست) (راست) (هدف)

۶. تحلیل محیط لغزنده (Slippery Environment)

تغییرات در احتمالات انتقال: در این حالت، محیط تصادفی (Stochastic) می‌شود. وقتی عامل فرمان حرکت به یک جهت (مثلاً «بالا») را می‌دهد، با احتمال $2/3$ واقعاً به بالا می‌رود و با احتمال $1/3$ در یکی از جهتهای مجاز دیگر می‌لغزد.

تاثیر بر روند همگرایی:

۱. **سرعت همگرایی:** لغزنده شدن محیط باعث افزایش چشمگیر واریانس در پاداش‌ها می‌شود. در نتیجه، نمودار همگرایی نوسانات بسیار بیشتری خواهد داشت و تعداد اپیزودهای بسیار بیشتری (مثلاً بیش از ۱۰۰۰۰ اپیزود) نیاز است تا مقادیر Q به ثبات برسند.
۲. **سیاست نهایی عامل (تغییر رفتار):** در محیط قطعی، عامل از کنار چاله‌ها با اطمینان عبور می‌کرد. اما در محیط لغزنده، عامل متوجه می‌شود که حتی اگر عمل درستی انتخاب کند، احتمال افتادن در چاله وجود دارد. بنابراین، سیاست نهایی هر دو الگوریتم (مخصوصاً SARSA) به شدت **محافظه‌کارانه** می‌شود. عامل یاد می‌گیرد که برای رفتن به هدف، تا جای ممکن به دیوارها بچسبد یا از مسیرهایی استفاده کند که در صورت لغزش، به جای چاله به یک دیوار امن برخورد کند (دیوار باعث ماندن در همان خانه می‌شود که بهتر از مرگ در چاله است).

تحلیل چالش پاداش پراکنده (Sparse Reward) و فرآیند اصلاح همگرایی

در اولین مرحله از پیاده‌سازی الگوریتم‌های Q-Learning و SARSA بر روی محیط دریاچه یخ‌زده (Frozen Lake) با نرخ اکتشاف ثابت ($\epsilon = 0.1$), نمودار مجموع و میانگین پاداش عامل‌ها در تمام ۲۰۰۰ اپیزود کاملاً بر روی عدد صفر باقی ماند و سیستم دچار عدم همگرایی مطلق گردید. پس از بررسی ماتریس‌های ارزش (Q-table)، علل تئوریک و فنی این رخداد به شرح زیر استخراج شد:

۱. **مسئله پاداش پراکنده (Sparse Reward Problem):** در این محیط، عامل تنها در صورت موفقیت کامل و رسیدن به خانه هدف (خانه ۱۶) پاداش $+1$ دریافت می‌کند و در سایر خانه‌ها (محیط‌های امن یخ‌زده) پاداش صفر است. زمانی که نرخ اکتشاف (ϵ) از ابتدا روی یک مقدار کوچک فیکس باشد، عامل فضای کافی برای جستجوی تصادفی ندارد. در نتیجه، احتمال اینکه بتواند به صورت کاملاً تصادفی زنجیره‌ای از ۷ الی ۸ حرکت درست را بدون افتادن در چاله‌ها طی کند تا به هدف برسد، نزدیک به صفر خواهد بود و عامل هرگز سیگنال پاداش مثبت را برای به‌روزرسانی زوایای ماتریس Q دریافت نمی‌کند.
۲. **محبوس شدن در حلقه‌های بی‌نهایت (Infinite Loop & Wall Collisions):** به دلیل مقداردهی اولیه ماتریس Q با اعداد صفر، در اپیزودهای اولیه که هنوز پاداشی کسب نشده، عامل در هر حالت اولین اکشن با بیشترین مقدار (که به صورت پیش‌فرض حرکت به «بالا» است) را انتخاب می‌کند. در ردیف اول شبکه، حرکت به سمت بالا باعث برخورد با دیواره و ماندن عامل در جای خود می‌شود. بدون در نظر گرفتن محدودیت گام، اپیزود فاقد وضعیت پایانی (Terminal State) شده و عامل تا ابد در یک حلقه بی‌نهایت گیر می‌افتد.

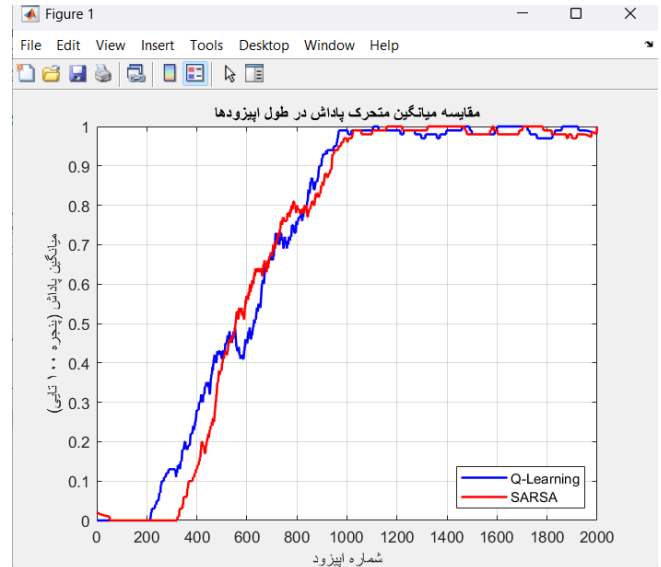
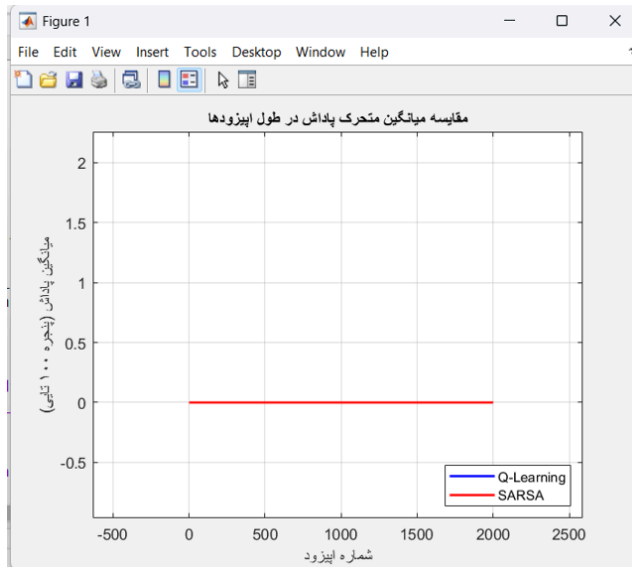
راهکار پایداری و اصلاح ساختار یادگیری

برای رفع این اختلال کلاسیک در یادگیری تقویتی، دو مکانیزم اساسی به الگوریتم اضافه شد:

- **کاهش تدریجی نرخ اکتشاف (ϵ - decay):** مقدار ϵ در اپیزودهای ابتدایی برابر با ۱.۰ (اکتشاف ۱۰۰٪ تصادفی) تنظیم شد تا عامل بدون وابستگی به ماتریس Q ، تمام ابعاد محیط را جستجو (Explore) کند و مسیر هدف را بیابد. سپس به تدریج و با پیشرفت اپیزودها، این نرخ کاهش یافت تا عامل به سمت بهره‌برداری (Exploitation) از دانش کسب‌شده حرکت کند.
- **قید حداکثر گام زمانی (Max Steps Limit):** سقف ۱۰۰ گام برای هر اپیزود تعریف شد تا در صورت محبوس شدن عامل در حلقه‌های تکراری، اپیزود قطع شده و محیط ریست شود.

نتیجه نهایی اصلاحات:

همان‌طور که در نمودار نهایی (شکل مربوطه) مشاهده می‌شود، با اعمال این تغییرات فازهای یادگیری به درستی شکل گرفتند. سیستم پس از طی کردن فاز اکتشاف نوسانی در ۲۰۰ اپیزود اول، یک روند صعودی شدید را آغاز کرده و در حدود اپیزود ۱۰۰۰، هر دو الگوریتم با موفقیت به پاداش پایدار ۱ (موفقیت ۱۰۰٪ در رسیدن به هدف) همگرا شدند.



خلاصه کل کارهایی که انجام شده:

۱. بخش رباتیک (سینماتیک و کنترل)

- ربات **SCARA**: درجات آزادی و آرایش مفاصل را مشخص کردیم، جدول DH را تشکیل دادیم و ماتریس‌های انتقال A_i را استخراج کردیم. در نهایت سینماتیک مستقیم را در متلب کدنویسی و به صورت عددی ارزیابی کردیم.
- ربات دو لینکی (**DOF-2**): تفاوت فضای کاری و مفصلی را بررسی کردیم و معادلات سینماتیک مستقیم و معکوس (روش هندسی) را به دست آوردیم.
- شبیه‌سازی و کنترل: توابع **FK_2R** و **IK_2R** را در متلب نوشتیم (با در نظر گرفتن محدودیت فضای کاری برای جلوگیری از تکیگی). سپس روند پیاده‌سازی در سیمولینک، اتصال بلوک‌ها برای ایجاد تبدیل همانی، تأثیر دینامیک موتورهای (تاخیر و تابع تبدیل) و تنظیم **PID** را بررسی کردیم. در نهایت استراتژی‌های کنترل در فضای کاری و فضای مفصلی را مقایسه کردیم و نشان دادیم چرا کنترل مفاصل در صنعت پایدارتر و کاربردی‌تر است.

۲. بخش یادگیری تقویتی (RL)

- مفاهیم تئوری و **MDP**: چالش اکتشاف/بهره‌برداری را تحلیل کردیم. برای ربات **Pick and Place**، ماز و شطرنج، سیگنال پاداش و تعریف حالت مناسب (**MDP**) ارائه دادیم و تفاوت تسک‌های اپیزودیک و پیوسته را با مثال مشخص کردیم.

- **ریاضیات یادگیری TD:** تابع ارزش را برای یک دنباله مشخص به صورت دستی محاسبه کردیم و دو اثبات مهم ریاضی (صعودی بودن ارزش با پاداش منفی و برابری خطای مونت کارلو با خطای TD تخفیف یافته) را انجام دادیم.
- **Frozen Lake:** محیط شبکه 4×4 دریاچه یخزده را به همراه الگوریتم‌های Q-Learning و SARSA در متلب پیاده‌سازی کردیم. در اجرای اول با چالش یادگیری (پاداش پراکنده و گیر کردن در حلقه بی‌نهایت) مواجه شدیم که آن را با اضافه کردن کاهش تدریجی اکتشاف ($\epsilon - Decay$) و محدودیت گام برطرف کردیم. در نهایت، با اجرای موفق کد، نمودار همگرایی بسیار زیبایی گرفتیم و تفاوت رفتار این دو الگوریتم (Off-policy در برابر On-policy) را با مثال Cliff Walking و همچنین تاثیر محیط لغزنده تحلیل کردیم.